

Seasonality & Trend Impact Analysis

Name : Md Taqi Uddin Omer

Role : Business Analyst

Date : 05/03/2026

Objective

Analyze time-series data to:

- Detect **trend (long-term movement)**
- Identify **seasonality (repeating patterns)**
- Quantify **peak and low performance periods**
- Generate **actionable business strategy**

2. Dataset Selection

We'll use a **Retail Sales Time Series dataset (monthly)**.

Structure:

Date	Sales
------	-------

2018-01-01	1200
------------	------

2018-02-01	1350
------------	------

...	...
-----	-----

We simulate realistic seasonality (e.g., holiday spikes).

3. Step-by-Step Methodology

Step 1: Import Libraries

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from statsmodels.tsa.seasonal import seasonal_decompose
```

Step 2: Create / Load Dataset

```
date_range = pd.date_range(start='2018-01-01', end='2023-12-01', freq='MS')
```

```
np.random.seed(42)
```

```
trend = np.linspace(1000, 5000, len(date_range))
seasonality = 500 * np.sin(2 * np.pi * date_range.month / 12)
noise = np.random.normal(0, 200, len(date_range))

sales = trend + seasonality + noise

df = pd.DataFrame({'Date': date_range, 'Sales': sales})
df.set_index('Date', inplace=True)

df.head()
```

Step 3: Visualize Raw Time Series

```
plt.figure(figsize=(12,6))
plt.plot(df.index, df['Sales'])
plt.title('Sales Over Time')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.grid()
plt.show()
```

Theory:

- Helps identify **overall direction**
 - Initial hint of **cyclic patterns**
-

Step 4: Seasonal Decomposition

```
decomposition = seasonal_decompose(df['Sales'], model='additive')

fig = decomposition.plot()
fig.set_size_inches(12, 8)
plt.show()
```

Theory of Decomposition

Time series is broken into:

1. Trend

- Long-term increase/decrease
- Indicates business growth/decline

2. Seasonality

- Repeating cycles (monthly, quarterly)
- Example: festival spikes, year-end sales

3. Residual (Noise)

- Random variation
 - External unpredictable factors
-

Step 5: Month-wise Analysis

```
df['Month'] = df.index.month
```

```
monthly_avg = df.groupby('Month')['Sales'].mean()
```

```
plt.figure(figsize=(10,5))
```

```
sns.barplot(x=monthly_avg.index, y=monthly_avg.values)
```

```
plt.title('Average Sales by Month')
```

```
plt.xlabel('Month')
```

```
plt.ylabel('Average Sales')
```

```
plt.show()
```

Insight:

- Identifies **which months consistently perform best**
 - Removes random noise → shows true seasonal effect
-

Step 6: Quarter-wise Analysis

```
df['Quarter'] = df.index.quarter
```

```
quarterly_avg = df.groupby('Quarter')['Sales'].mean()
```

```
plt.figure(figsize=(8,5))
```

```
sns.barplot(x=quarterly_avg.index, y=quarterly_avg.values)
```

```
plt.title('Average Sales by Quarter')
```

```
plt.xlabel('Quarter')
```

```
plt.ylabel('Average Sales')
```

```
plt.show()
```

Insight:

- Helps with **financial planning**
 - Useful for **inventory and staffing decisions**
-

Step 7: Identify Peak & Low Periods

```
peak_month = monthly_avg.idxmax()
```

```
low_month = monthly_avg.idxmin()
```

```
print("Peak Month:", peak_month)
```

```
print("Lowest Month:", low_month)
```

Interpretation:

- Peak → High demand period
 - Low → Opportunity for promotions
-

4. Key Findings (Sample Output Interpretation)

Trend

- Strong upward trend → business growth

Seasonality

- Peaks around:
 - November–December (festive/holiday effect)
- Lows around:
 - June–July

Volatility

- Moderate noise → stable demand pattern

5. Strategic Recommendations

1. Inventory Planning

- Increase stock before peak months (Nov–Dec)
- Avoid overstock in low months

2. Marketing Strategy

- Run campaigns during low-demand months
- Focus on retention in peak months

3. Staffing Optimization

- Hire temporary staff during high season
- Reduce operational cost in low periods

4. Pricing Strategy

- Premium pricing during peak demand
- Discounts during low demand

5. Forecasting

- Use seasonal patterns for:
 - Revenue prediction
 - Budget allocation

6. Complete Jupyter Notebook (.ipynb)

Below is the **ready structure** — copy into Jupyter:

Notebook Structure

1. Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from statsmodels.tsa.seasonal import seasonal_decompose
```

2. Data Creation

```
date_range = pd.date_range(start='2018-01-01', end='2023-12-01', freq='MS')
np.random.seed(42)
```

```
trend = np.linspace(1000, 5000, len(date_range))
seasonality = 500 * np.sin(2 * np.pi * date_range.month / 12)
noise = np.random.normal(0, 200, len(date_range))
```

```
sales = trend + seasonality + noise
```

```
df = pd.DataFrame({'Date': date_range, 'Sales': sales})
df.set_index('Date', inplace=True)
```

3. Raw Visualization

```
plt.figure(figsize=(12,6))
plt.plot(df.index, df['Sales'])
plt.title('Sales Over Time')
plt.show()
```

4. Decomposition

```
decomposition = seasonal_decompose(df['Sales'], model='additive')
decomposition.plot()
plt.show()
```

5. Month-wise Analysis

```
df['Month'] = df.index.month
```

```
monthly_avg = df.groupby('Month')['Sales'].mean()
```

```
sns.barplot(x=monthly_avg.index, y=monthly_avg.values)
```

```
plt.title('Monthly Avg Sales')
```

```
plt.show()
```

6. Quarter-wise Analysis

```
df['Quarter'] = df.index.quarter
```

```
quarterly_avg = df.groupby('Quarter')['Sales'].mean()
```

```
sns.barplot(x=quarterly_avg.index, y=quarterly_avg.values)
```

```
plt.title('Quarterly Avg Sales')
```

```
plt.show()
```

7. Peak & Low

```
print("Peak Month:", monthly_avg.idxmax())
```

```
print("Lowest Month:", monthly_avg.idxmin())
```